

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	JASMITHA R	Reg. No.:	192524295
Section / Batch:	AI&DS	Date:	27/08/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

Code of Conduct

I JASMITHA R(192524295)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: JASMITHA R

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%				
Constraint Analysis	25%				
Solution Development	25%				
Application of Concepts	15%				
Justification	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Hamiltonian Path Problem (NP-Complete / Backtracking Problem)

1. Problem Statement

The Hamiltonian Path Problem is to determine whether a graph contains a path that visits every vertex exactly once. A valid path must follow the edges of the graph without revisiting any vertex. Since this is an NP-Complete problem, a backtracking approach is commonly used to explore all possible paths. The problem is named after Sir William Rowan Hamilton, who studied similar path-based puzzles on the dodecahedron graph in the nineteenth century.

A closely related problem is the Hamiltonian Cycle Problem, which additionally requires that the last vertex in the path be connected back to the starting vertex, forming a closed loop. The Hamiltonian Path Problem is considered a relaxed version of the cycle problem because it does not require the path to close back on itself.

2. Input

- A graph $G(V, E)$ where:
 - V = Set of vertices (cities)
 - E = Set of edges (connections)
- The graph may be represented using an adjacency matrix or an adjacency list, and it may be directed or undirected depending on the problem instance.
- Optionally, a specific starting vertex may be given; otherwise, every vertex is tried as a possible starting point.

3. Output

- True – A Hamiltonian Path exists.
- False – No Hamiltonian Path exists.

When the output is True, the algorithm also reports the actual sequence of vertices that forms the valid Hamiltonian Path, since simply knowing that one exists is often insufficient for practical applications such as routing or scheduling.

4. Constraints

- Every vertex must be visited exactly once.
- Consecutive vertices in the path must be connected by an edge.
- No vertex can be revisited.
- The path must contain all vertices.
- The path does not need to return to the starting vertex (unlike the Hamiltonian Cycle Problem).
- If the graph is disconnected, no Hamiltonian Path can exist, since some vertices would be unreachable from others.
- A graph with an isolated vertex of degree zero (and more than one vertex overall) cannot contain a Hamiltonian Path.

5. Problem Analysis

The Hamiltonian Path problem has a very large search space because every vertex can be a possible starting point, and each step may have

several choices. In the worst case, where the graph is fully connected (a complete graph), the number of possible orderings of vertices is $N!$, since after choosing a starting vertex there are $(N-1)$ choices for the next vertex, $(N-2)$ choices for the one after, and so on.

Invalid paths are discarded using constraint checking, while backtracking explores other possibilities until a valid path is found or all options are exhausted. The key insight that makes backtracking more efficient than brute-force generation of all $N!$ permutations is that a partial path is checked for validity at every step; if the next vertex is not adjacent to the current vertex, or has already been visited, the algorithm prunes that branch immediately instead of continuing to build an invalid sequence.

The density of the graph directly affects the size of the practical search space. Sparse graphs, where each vertex has few neighbors, prune very quickly because most vertex pairs are not directly connected. Dense graphs, especially complete graphs, offer many more valid extensions at each step, which increases the effective branching factor and makes the search significantly slower.

6. Backtracking Pseudocode

Algorithm HamiltonianPath(Graph G)

Input : Graph $G(V,E)$

Output: True if Hamiltonian Path exists, otherwise False

For each vertex v in G do

```

    Mark all vertices as unvisited
    Path[1]  $\leftarrow$  v

    If Backtrack(v,1) = True then
        Return True
    End If
End For

Return False
Function Backtrack(Current, Count)

    Mark Current as visited

    If Count = TotalVertices then
        Print Path
        Return True
    End If

    For each Neighbor of Current do

        If Neighbor is not visited AND
           Edge(Current, Neighbor) exists then

            Path[Count+1]  $\leftarrow$  Neighbor

            If Backtrack(Neighbor, Count+1) = True then
                Return True
            End If

        End If

    End For

End For

```

Mark Current as unvisited

Return False

7. Algorithm

1. Read the graph as input.
2. Choose each vertex as the starting point.
3. Mark the current vertex as visited.
4. Visit every unvisited adjacent vertex recursively.
5. Check whether all vertices have been visited.
6. If yes, return True.
7. If no valid path is found, backtrack by unmarking the vertex.
8. Repeat until all possibilities are explored.
9. Return False if no Hamiltonian Path exists.

8. Constraint Checking

- Ensure every vertex is visited only once.
- Ensure movement is only through connected vertices.
- Reject paths containing repeated vertices.
- Accept the solution only when all vertices are included.
- Verify at each recursive call that the count of visited vertices matches the position in the path, so that no vertex is skipped or double-counted.

- Discard a branch immediately if the current vertex has no unvisited adjacent vertex and the path is still incomplete, since this indicates a dead end.

9. Pruning Techniques

Although the base algorithm already discards invalid neighbors, additional pruning strategies can be layered on top to make the search considerably faster on larger graphs:

- Degree-based pruning: if any unvisited vertex has degree zero with respect to the remaining unvisited vertices, no Hamiltonian Path can be completed through it, so the branch is abandoned early.
- Connectivity pruning: before starting the search, check that the graph is connected; a disconnected graph can never contain a Hamiltonian Path, so the search can be skipped entirely.
- Warnsdorff-like heuristic ordering: neighbors with fewer remaining unvisited connections are tried first, since they are more likely to become dead ends later and should be resolved earlier in the search.
- Symmetry pruning: for undirected graphs, a path and its reverse represent the same Hamiltonian Path, so fixing the direction of travel from the lexicographically smaller endpoint avoids exploring duplicate mirrored paths.

- Remaining vertex count check: if the number of remaining unvisited vertices exceeds the number of vertices still reachable from the current vertex through unvisited neighbors, the branch is pruned.

10. Solution Justification

Backtracking systematically explores every possible path while eliminating invalid paths early using constraint checking. It guarantees finding a Hamiltonian Path if one exists. Although computationally expensive, it is a complete and correct approach for solving NP-Complete problems.

Completeness follows from the fact that every recursive call examines all unvisited neighbors of the current vertex, so no valid extension of the path is ever overlooked. A branch is pruned only when it has been proven that the branch cannot possibly complete into a Hamiltonian Path (for example, the next vertex is not adjacent or has already been visited). Because pruning never removes a potentially valid branch, the algorithm will always find an existing Hamiltonian Path, and it will correctly report False only after every starting vertex and every combination of valid extensions has been exhausted.

11. Complexity Analysis

Time Complexity

- $O(N!)$

- The algorithm may examine all possible vertex arrangements in the worst case, which occurs most notably in dense or complete graphs where nearly every vertex is adjacent to every other vertex, leaving little opportunity for early pruning.

Space Complexity

- $O(N)$
- Stores the visited array, recursion stack, and current path. The recursion depth can reach N in the worst case, since the path can grow to include all vertices before backtracking begins.

It is worth noting that the Hamiltonian Path Problem, like the Hamiltonian Cycle Problem, belongs to the NP-Complete complexity class. Membership in NP follows because a proposed path can be verified in $O(N)$ time by checking that all vertices appear exactly once and that consecutive vertices are connected by an edge. NP-Hardness follows from a polynomial-time reduction from the well-known Hamiltonian Cycle Problem, which is itself NP-Complete, meaning no polynomial-time algorithm is currently known for solving the general case.

12. Advantages

- Finds the correct solution if one exists.
- Simple recursive implementation.
- Suitable for small and medium-sized graphs.
- Uses pruning to avoid invalid paths.

- Can be easily adapted to report all Hamiltonian Paths instead of stopping at the first one found.
- Requires only linear additional memory beyond the graph representation itself.

13. Disadvantages

- High execution time for large graphs.
- Not suitable for dense graphs with many vertices.
- Exponential search space.
- Performance is highly sensitive to the order in which neighbors are explored, and a poor ordering heuristic can lead to unnecessarily long search times.
- Does not scale well beyond a few tens of vertices for dense or adversarially constructed graphs.

14. Applications

- Route planning and navigation.
- Network design.
- Robot path planning.
- Circuit board design.
- DNA sequencing.
- Logistics and transportation.

- Scheduling problems.
- Puzzle solving, such as the classic Knight's Tour problem on a chessboard, which is itself a special case of the Hamiltonian Path Problem.
- Manufacturing process ordering, where each machine or station must be visited exactly once with minimal reconfiguration.

15. Example Walkthrough

Consider a graph with four vertices A, B, C, and D, and edges A-B, B-C, C-D, and D-A, along with an additional edge A-C. Starting the search at vertex A, the algorithm marks A as visited and tries its neighbors in order: B, D, and C. Suppose it first moves to B (Path: A, B). From B, the only unvisited neighbor is C, so the algorithm moves to C (Path: A, B, C). From C, the unvisited neighbors are D (since A is already visited), so the algorithm moves to D (Path: A, B, C, D). At this point, all four vertices have been visited, and Count equals TotalVertices, so the algorithm returns True with the Hamiltonian Path $A \rightarrow B \rightarrow C \rightarrow D$.

If the edge B-C had not existed, the algorithm would have reached a dead end after visiting B, since B would have no unvisited neighbors left to extend the path. In that case, the algorithm would backtrack, unmark B as visited, and try the next neighbor of A, continuing the search until either a valid path is found through a different route or all possibilities from every starting vertex have been exhausted.

16. Comparison with the Hamiltonian Cycle Problem

The Hamiltonian Path Problem is closely related to, but distinct from, the Hamiltonian Cycle Problem. A Hamiltonian Cycle requires that, in addition to visiting every vertex exactly once, the final vertex in the path must also be connected back to the starting vertex, forming a closed loop. Every Hamiltonian Cycle contains a Hamiltonian Path (simply remove the final closing edge), but not every Hamiltonian Path can be extended into a Hamiltonian Cycle, since the last vertex may not be adjacent to the first. Both problems share the same NP-Complete classification and can be solved using nearly identical backtracking algorithms, differing only in the final check performed once all vertices have been visited.

17. Conclusion

The Hamiltonian Path Problem determines whether every vertex in a graph can be visited exactly once. A backtracking algorithm recursively explores all possible paths while checking constraints such as vertex visitation and edge connectivity. The algorithm guarantees correctness but has a worst-case time complexity of $O(N!)$, making it practical mainly for smaller graphs.

Through the use of pruning techniques such as degree-based checks, connectivity verification, and heuristic neighbor ordering, the practical performance of the backtracking approach can be substantially improved, even though the worst-case complexity remains exponential. As with many NP-Complete problems, the Hamiltonian Path Problem illustrates

the trade-off between guaranteed correctness and computational feasibility, and it remains an important benchmark problem for studying graph algorithms, constraint satisfaction, and the broader theory of computational complexity.